Dynamic Bending Test Rig

12.500 Hz is the required actuator bandwidth, and the requried force is 16.8787 lbf as per section 2.2 . Assuming that you are interested in replicating the force (rather than displacement)

1 Actuator Sizing from Shear Forces MATLAB (outdated)

Now, I have a force/time graph Now I need to get a bandwidth out of it.

Quickest ramp looks to be

```
timeForceRamp = 5.56-5.52
```

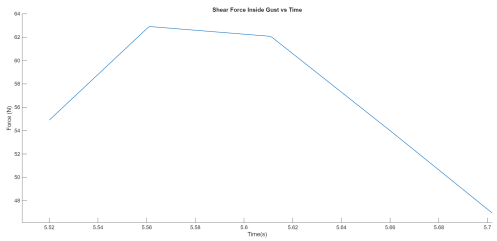
```
timeForceRamp =
0.0400
```

```
freqTimeRamp = 1/timeForceRamp
```

```
freqTimeRamp =
25.0000
```

```
deltaForce = 63 - 55
```

```
deltaForce =
8
```



2.1 Actuator Sizing from Gust Velocity Profile & Ian's Calculations

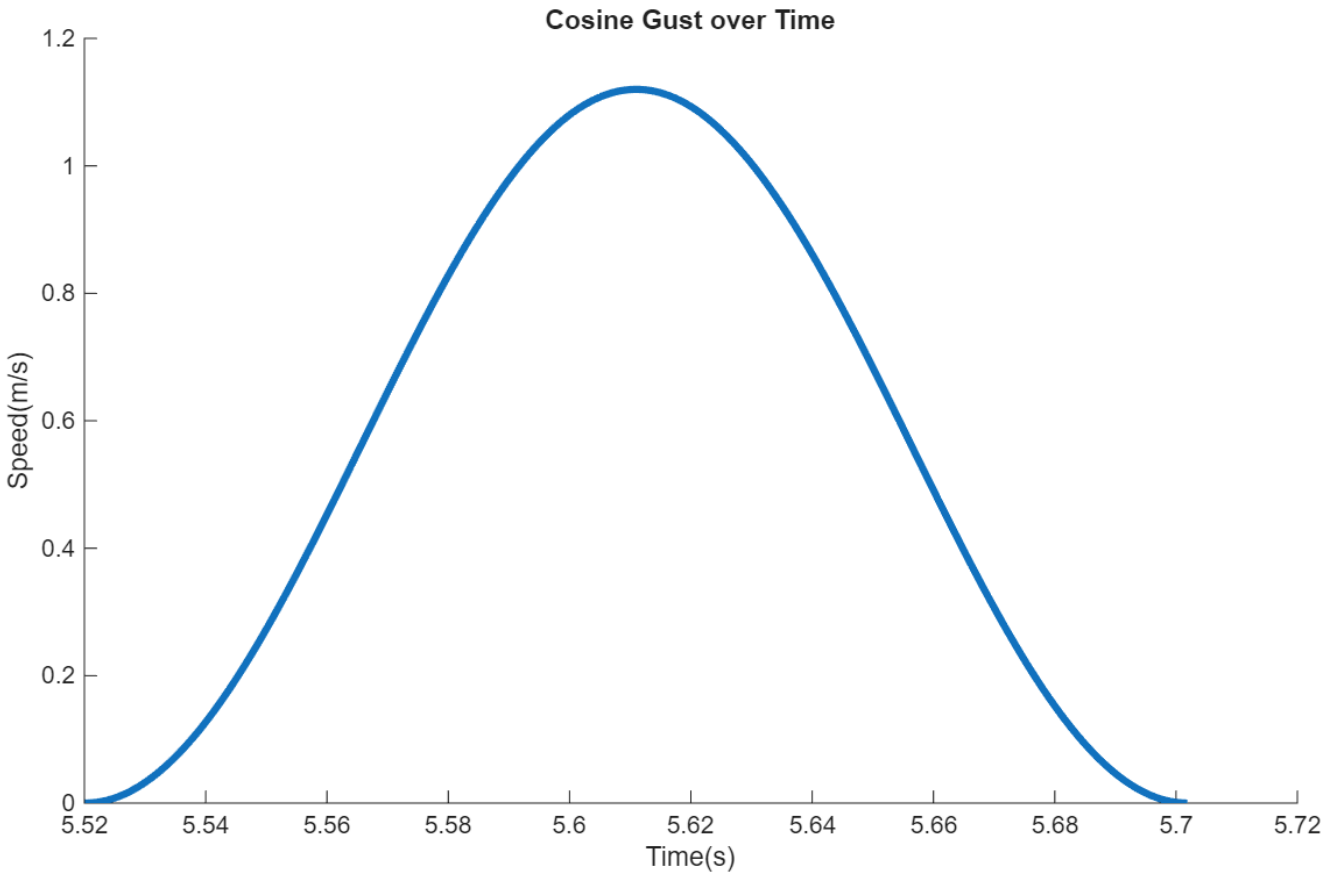
From Ian's Shear Calculations (at 1.7 AoA):

https://docs.google.com/spreadsheets/d/1YpCOBGyIhGbyl9DqY5GLDXi1CcHhjao9K5FwuqBK1_Q/edit?usp=sharing

		2.744895879	1.18593478	-63.05414586	86.91810389
	m5 inertial load	2.75	-91.44732192	-154.5014678	86.12950966
		2.820098507	0.07498686142	-154.4264809	75.30444392
		2.895301135	0.4359773457	-153.9905036	63.72395334
		2.970503763	0.4987899236	-153.4917136	52.18097307
		3.045706391	0.5478739359	-152.9438397	40.67919436
		3.12090902	0.6578691834	-152.2859705	29.22688914
		3.196111648	6.472251167	-145.8137194	18.26131422
		3.271314276	56.66729428	-89.14642508	11.55726876
		3.346516904	106.8224795	17.67605444	12.88655451
		3.421719532	146.274175	163.9502294	25.21604265
	m6 inertial load	3.43	-274.9276715	-110.9774421	24.29709753
		3.49692216	73.6050986	-37.37234349	21.79605956
		3.572124789	21.27224074	-16.10010275	20.58528952
		3.647327417	-2.184223439	-18.28432619	19.21026014
		3.722530045	44.74627641	26.46195022	21.20026834
	m7 inertial load	3.83	-26.45747998	0.004470241051	21.20074876

`164 N` is the max shear, which is about `36.8687 lbf`

From MATLAB code for the Cosine Gust:



The time this occurs at is around 5.52s to 5.7s, which results in a 0.18s time for the gust

Apply a factor of safety of two, so that there is a 0.09s cycling requirement for the cosine gust.

This results in the required bandwidth being as follows:

$$f = \frac{1}{.09}$$

```
fBW1 = 1/.09 % in Hz
```

```
fBW1 =
11.1111
```

2.2 Actuator Sizing Based Off OpenRocket AoA & Ian's Calculations

Take the AoA profile from OpenRocket and multiply it by Ian's peak shear at peak angle of attack to transform the AoA curve to a shear curve

```
%% OpenRocket CSV -> Angle of Attack vs Time & Altitude (robust)
% - Handles OpenRocket comment lines (# ...)
% - Uses the detected header line for variable names
% - Robustly finds Time / Altitude / AoA columns
% - Forces vectors to be column vectors to avoid table row-size errors

clear; clc;

filePath = "OpenRocket.csv";   % or full path if needed

%% 1) Find the header line + first data line (first non-# line)
fid = fopen(filePath,'r');
assert(fid ~= -1, "Could not open file: %s", filePath);

headerLine = "";
dataStartLine = NaN;

lineNum = 0;
while ~feof(fid)
    lineNum = lineNum + 1;
    ln = fgetl(fid);
    if ~ischar(ln), break; end
    lnTrim = strtrim(ln);
```

```
    % OpenRocket commonly writes: "# Time (s),Altitude (m),..."
    if startsWith(lnTrim, "#") && contains(lnTrim, "Time") && contains(lnTrim,
"Altitude")
        headerLine = erase(lnTrim, "#");
        headerLine = strtrim(headerLine);
    end

    % First actual data row is typically the first non-# line
    if ~startsWith(lnTrim, "#") && ~isempty(lnTrim) && isnan(dataStartLine)
        dataStartLine = lineNum;
        break;
    end
end
fclose(fid);

assert(~isempty(headerLine), "Couldn't find the OpenRocket header row (the '#
Time...,Altitude...' line).");
assert(~isnan(dataStartLine), "Couldn't find the first data row (first non-#
line).");

%% 2) Build MATLAB-friendly variable names from the header
rawNames = split(headerLine, ",");
rawNames = strtrim(rawNames);

% makeValidName removes spaces/symbols so e.g. "Time (s)" -> "Times"
varNames = matlab.lang.makeValidName(rawNames, "ReplacementStyle","delete");

%% 3) Read numeric data from the first data line onward
opts = delimitedTextImportOptions("NumVariables", numel(varNames));
opts.DataLines = [dataStartLine, Inf];
opts.Delimiter = ",";
opts.VariableNames = varNames;
opts.VariableTypes = repmat("double", 1, numel(varNames));
opts.ExtraColumnsRule = "ignore";
opts.EmptyLineRule = "read";

T = readtable(filePath, opts);

%% 4) Find the Time / Altitude / AoA columns (works with unit-suffixed names)
names = string(T.Properties.VariableNames);

% exact-match candidates (common OpenRocket outputs after makeValidName)
timeCandidates = ["Times","Time_s","Time","time","TimeSeconds"];
altCandidates  = ["Altitudem","Altitude_m","Altitude","altitude"];
aoaCandidates  =
["AngleOfAttack","AngleOfAttack_deg","Angle_of_attack","AoA","alpha"];

timeVar = intersect(timeCandidates, names, "stable");
altVar  = intersect(altCandidates,  names, "stable");
```

```
aoaVar  = intersect(aoaCandidates,  names, "stable");

% fallback: contains (case-insensitive)
if isempty(timeVar)
    timeVar = names(contains(lower(names), "time"));
end
if isempty(altVar)
    altVar = names(contains(lower(names), "altitude"));
end
if isempty(aoaVar)
    aoaVar = names(contains(lower(names), "angleofattack") | contains(lower(names),
"aoa"));
end

assert(~isempty(timeVar), "Couldn't find Time column. Columns found: %s",
strjoin(names, ", "));
assert(~isempty(altVar),  "Couldn't find Altitude column. Columns found: %s",
strjoin(names, ", "));
assert(~isempty(aoaVar),  "Couldn't find Angle of attack column. Columns found:
%s", strjoin(names, ", "));

% pull data
t   = T.(timeVar(1));
alt = T.(altVar(1));
aoa = T.(aoaVar(1));

%% 5) Force all to be column vectors (fixes 'same number of rows' table error)
t   = t(:);
alt = alt(:);
aoa = aoa(:);

% assert(numel(t)==numel(alt) && numel(t)==numel(aoa), ...
%     "Lengths mismatch: t=%d, alt=%d, aoa=%d", numel(t), numel(alt), numel(aoa));
%
% AoA_Time_Altitude = table(t, alt, aoa, ...
%     "VariableNames", ["Time_s","Altitude_m","AngleOfAttack_deg"]);
%
% % 6) Display + plots
% disp("Detected columns:");
% disp("  Time:     " + timeVar(1));
% disp("  Altitude: " + altVar(1));
% disp("  AoA:      " + aoaVar(1));
%
% disp(AoA_Time_Altitude(1:min(10,height(AoA_Time_Altitude)), :));
%
% figure;
% plot(AoA_Time_Altitude.Time_s, AoA_Time_Altitude.AngleOfAttack_deg);
% xlabel("Time (s)"); ylabel("Angle of attack (deg)");
% grid on; title("Angle of Attack vs Time");
%
```

```
% figure;
% scatter(AoA_Time_Altitude.Altitude_m, AoA_Time_Altitude.AngleOfAttack_deg, 10,
"filled");
% xlabel("Altitude (m)"); ylabel("Angle of attack (deg)");
% grid on; title("Angle of Attack vs Altitude");
```

Then to generate the graph for shear (from the max shear calculated via CFD at 1.7 AoA):

164 N ~ 36.6887 lbf

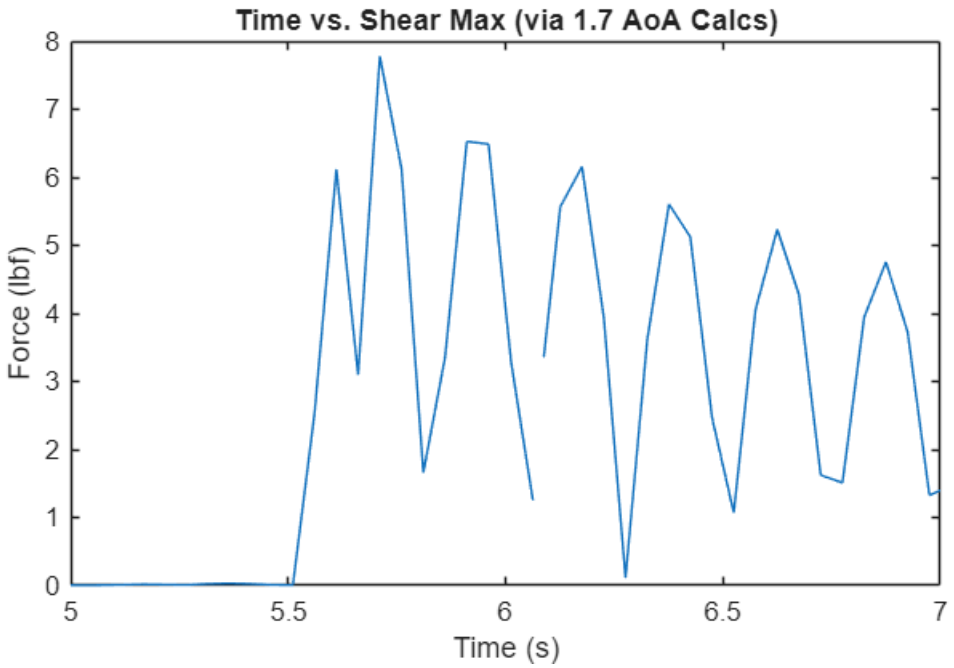
```
shearMax = 36.8687
```

```
shearMax =
36.8687
```

```
shearGraph = aoa*shearMax
```

```
shearGraph = 1241×1
10^3 ×
         0
         0
    0.0000
    0.0000
    0.0000
    0.0000
       NaN
    0.0000
    0.0000
    0.0000
    0.0000
    0.0000
    0.0000
    0.0000
    0.0000
     ⋮
```

```
figure
plot(t,shearGraph)
xlim([5, 7])
title("Time vs. Shear Max (via 1.7 AoA Calcs)")
xlabel("Time (s)"); ylabel("Force (lbf)")
```



Look athte 6.6 AoA

	2.74489588	5.7427833	-195.2332689	315.8000347
m5 inertial load	2.745	-332.3405003	-527.5737692	315.7451037
	2.82009851	2.66213073	-524.9116384	276.3250218
	2.89530114	2.83878607	-522.0728524	237.0637703
	2.97050376	3.18801036	-518.884842	198.0422707
	3.04570639	2.85598701	-516.028855	159.2355436
	3.12090902	4.55424976	-511.4746052	120.7713081
	3.19611165	19.2521009	-492.2225043	83.75488125
	3.27131428	174.474026	-317.7484783	59.85936
	3.3465169	346.922944	29.17446566	62.05335626
	3.42171953	482.646504	511.8209697	100.5436393
m6 inertial load	3.4345	-1004.016361	-492.1953916	94.25315083
	3.49692216	373.629416	-118.5659756	86.85200653
	3.57212479	131.519018	12.95304235	87.82610938
	3.64732742	19.3586972	32.31173955	90.25603717
	3.72253005	69.7030941	102.0148337	97.92782096
m7 inertial load	3.833	-96.78717484	5.22765881	98.50532017

524.9116 N ~ 118.00482242 lbf

```
shearMax = 118.00482242
```

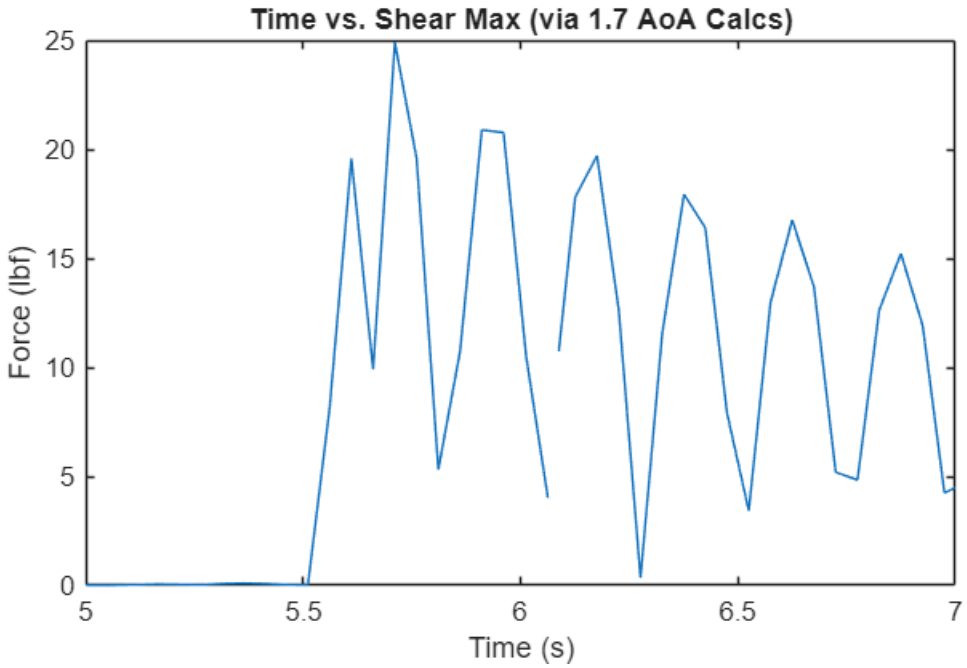
```
shearMax =
118.0048
```

```
shearGraph = aoa*shearMax
```

```
shearGraph = 1241×1
10^4 ×
         0
         0
    0.0000
```

```
    0.0000
    0.0000
    0.0000
       NaN
    0.0000
    0.0000
    0.0000
    0.0000
    0.0000
    0.0000
    0.0000
    0.0000
     ⋮
```

```
figure
plot(t,shearGraph)
xlim([5, 7])
title("Time vs. Shear Max (via 1.7 AoA Calcs)")
xlabel("Time (s)"); ylabel("Force (lbf)")
```



This amount of force looks to be too conservative.

Let's look at 1.7 AoA and get the limits to get a better look at the quickest ramp required:

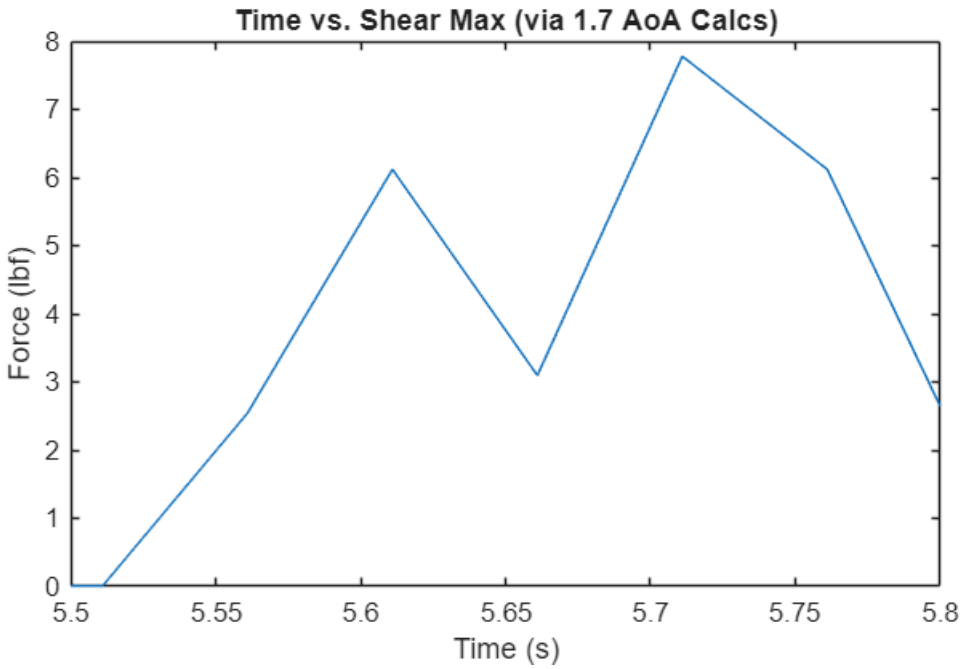
```
shearMax = 36.8687
```

```
shearMax =
36.8687
```

```
shearGraph = aoa*shearMax
```

```
shearGraph = 1241×1
10^3 ×
         0
         0
    0.0000
    0.0000
    0.0000
    0.0000
       NaN
    0.0000
    0.0000
    0.0000
    0.0000
    0.0000
    0.0000
    0.0000
    0.0000
      ⋮
```

```
figure
plot(t,shearGraph)
xlim([5.5 5.8])
title("Time vs. Shear Max (via 1.7 AoA Calcs)")
xlabel("Time (s)"); ylabel("Force (lbf)")
```



Time is at 5.64 to 5.72~

0.08s required for the cycle, and then bandwidth would be

```
FBW2 = 1/.08
```

```
FBW2 =
12.5000
```

12.500 Hz is the required actuator bandwidth, and the requried force is 16.8787 lbf

These calculations are more conservative than the previous guesstimate, so use this instead. The 2x required force from the 1.7 AoA is used in the above conclusion

3.1 Actuator Choice: Calculating how actuator maps to speed requirements (if we were interested in displacement)

A consideration I had was whether or not we wanted to replicate the displacement caused by the shear forces. To analyze what kind of actuator we would need if we wanted to replicate dispalcement(as well as force application), I applied calculations that assumed simple harmonic movement of a lienar actuator.

	2.74489579	1.1859348	-63.05414586	86.91810389
m5 inertial load	2.75	-91.44732192	-154.5014678	86.12950966
	2.820098507	0.07498686142	-154.4264809	75.30444392
	2.895301135	0.4359773457	-153.9905036	63.72395334
	2.970503763	0.4987899236	-153.4917136	52.18097307
	3.045706391	0.5478739359	-152.9438397	40.67919436
	3.12090902	0.6578691834	-152.2859705	29.22688914
	3.196111648	6.472251167	-145.8137194	18.26131422
	3.271314276	56.66729428	-89.14642508	11.55726876
	3.346516904	106.8224795	17.67605444	12.88655451
	3.421719532	146.274175	163.9502294	25.21604265
m6 inertial load	3.43	-274.9276715	-110.9774421	24.29709753
	3.49692216	73.6050986	-37.37234349	21.79605956
	3.572124789	21.27224074	-16.10010275	20.58528952
	3.647327417	-2.184223439	-18.28432619	19.21026014
	3.722530045	44.74627641	26.46195022	21.20026834
m7 inertial load	3.83	-26.45747998	0.004470241051	21.20074876

From the figure above(Ian's Calcualtions), grab the lever arm as the length form the nose cone to the point of max shear

```
leverArm = 3.417 % in m
```

```
leverArm =
3.4170
```

Using the radius, and recalling that for SHM:

X(t) =

```
% how to get displacement amplitude, using small angle aprox
% pick largest angle for this, 10 degrees. which equates to 0.174533 rad
strokeLength = leverArm *deg2rad(5) % in m
```

```
strokeLength =
0.2982
```

```
% peak velocity
velPeak = 2*pi * FBW2*strokeLength % in m/s
```

```
velPeak =
23.4197
```

$V_{max} = 2\pi fA$

```
% peak acceleration
accPeak = ((2*pi * FBW2)^2)*strokeLength% in m/s
```

```
accPeak =
1.8394e+03
```

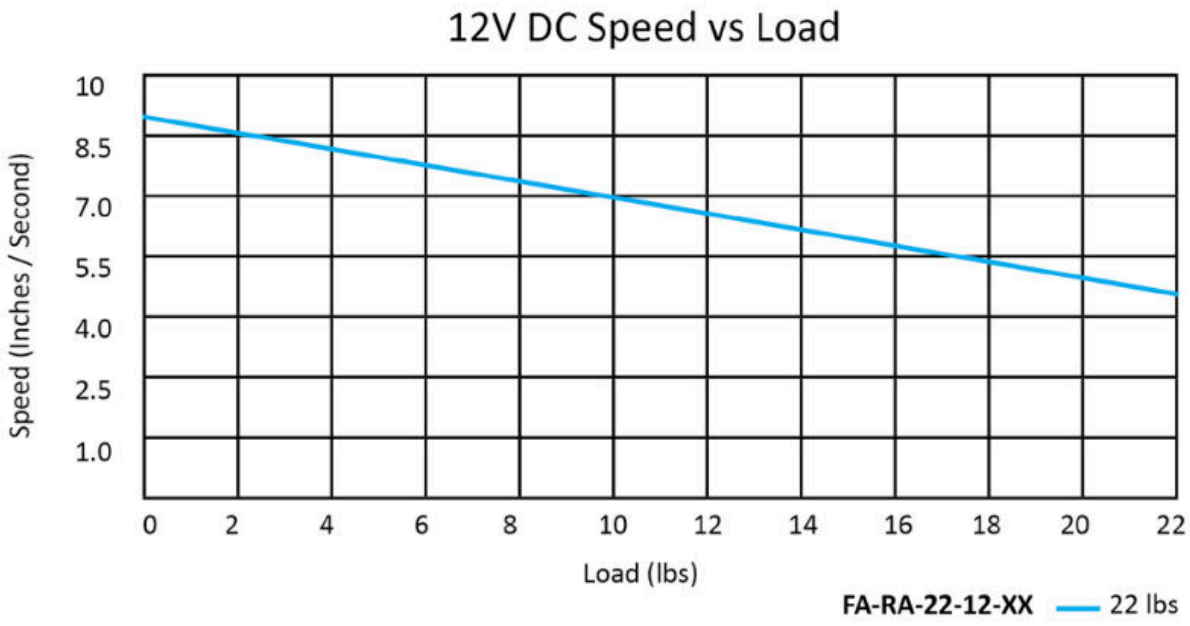
```
```

These values indicate that looking to replicate the displacement as well as the force profile would prove expensive, due to it requiring a peak velocity at 46.8395 m/s , and a peak acceleration of

3.2.1 Options for Actuators that Meet Requirements, HS Electric LA

Firgelli High Speed Linear Actuator

https://cdn-firgelliauto.sfo2.cdn.digitaloceanspaces.com/product-downloads/linear-actuator/firgelli-high-speed-linear-actuator-spec-sheet.pdf



Does not meet speed requirement. (7 in/s at 8 lbf) v_la = 0.185 m/s . This is way under velocity peak of 46.8395

```
% look at the speed requirement
v_la = 0.185
```

```
v_la =
0.1850
```

```
freqEstLA = v_la/(2*pi*strokeLength) % in Hz
```

```
freqEstLA =
0.0494
```

```

```

Waay too slow

3.2.2 Options for Acutators that Meet Requirements, Voice Coil LA

NCC15-24-090-1X
Non-Commutated DC Linear Actuator. Moving Coil Type. 9.0 lbs (40 N) Continuous Force, 27 lbs (120 N) Peak Force @ 10% duty, 1.5" (38.1 mm) Travel. OD = 2.38" 60.5 mm), Length at mid-stroke = 3.13" (79.5 mm), radial clearance on either side of coil = 0.063" (0.38 mm). Reference H2W drawing # 30-0984 Rev -.
Please note: This motor requires a customer-supplied bearing system for it to operate properly.
Price: 1 piece @ US$1,983 each

NON-COMM ACTUATOR SPECIFICATIONS		
Motor P/N	NCC15-24-090-1X	
Device Type	Moving Coil	
Stroke	1.50 in	38.1 mm
Radial Clearance	0.015 in	0.38 mm
Bearing Type	None Provided	
Moving Mass	0.75 lbs	340 grams
Total Mass	2.38 lbs	1080 grams
Resistance @ 20C	5.0 ohms	
Inductance @ 20C	2.8 mH	
Electrical Time Constant	0.56 msec	
Motor Constant	1.82 lbs/sqrt(watt)	8.1 N/sqrt(watt)
Force Constant	4.1 lbs/amp	18.1 N/amp
Back EMF	0.46 V/ips	18.1 V/m/sec
Force @ 100% Duty	9.0 lbs	40 N
Power @ 100% Duty	30 watts	
Current @ 100% Duty	2.2 amps	
Force @ 10% Duty	27 lbs	120 N
Power @ 10% Duty	220 watts	
Current @ 10% Duty	6.6 amps	

Looking at Force Requirements (for specified displacement)

```
forceConstant  = 4.1;
% calculate required amperage for force requirement
forceReq = 16.8787;

ampReq = 4.1*16.8787
```

```
ampReq =
69.2027
```

Additionally, at 10% duty meets the force requirement as 27lbs > 16.67 lbf

Looking at Peak Velocity Requirements (for specified displacement corresponding to Bandwidth via SHM)

Back EMF is noted to be 0.46 V/ips, and also 18.1 V/m/sec

```
% note that vel peak is in meters/s
BEMF = 18.1;
reqVfromBEMF = (velPeak*BEMF) % in volts
```

```
reqVfromBEMF =
423.8975
```

```
% look at the force constant


% calculate wattage requirement
powerReqVCLA = ampReq*reqVfromBEMF
```

```
powerReqVCLA =
2.9335e+04
```

```
% 29 kW requirement
```

Waaaay too slow